1 ScratchOS

ScratchOS is a minimal operating system written from scratch, in C and assembly, for the **ARM Cortex-M3**. It runs on the `mps2-an385` board emulated by QEMU and is built bottom-up: every mechanism that a real-time operating system such as FreeRTOS provides ready-made is implemented by hand, starting from the first instruction executed after reset.

> **WHY BUILD AN OS BY HAND**
>
> An RTOS hides its mechanisms behind an API. Knowing what happens underneath (how a task starts, what a context switch saves, why a critical section is needed) makes that API much easier to use correctly.

1.1 Design Choices

ScratchOS follows the choices typical of a small embedded OS, described in the introduction: a single image, a fixed set of tasks, and predictable behaviour.

Aspect	Choice in ScratchOS
Kernel architecture	Flat: startup code, kernel, drivers, and tasks linked into one image and one address space
Isolation	Added on top of the flat design: tasks run unprivileged, reach the kernel only through `svc`, and are confined to their partition by the MPU
Tasks	Fixed: compiled and linked together with the kernel, so no loader is needed
Memory	Static allocation: every address and size is decided at link time
Devices	Direct register access through drivers inside the kernel
Scheduling	Periodic SysTick tick, round-robin time slicing, and an idle task

1.2 From Reset to a Multitasking Kernel

ScratchOS is built as a chain of mechanisms, where each one depends on the previous ones. The chapters of this part follow the same order.

1. **Boot**: the vector table and `Reset_Handler` bring the CPU from reset to `main()`, preparing the stack, `.data`, and `.bss` (chapter *Boot and CPU Configuration*).
2. **Devices and interrupts**: drivers for the UART and the SysTick timer, first polled and then interrupt-driven through the NVIC (chapter *Devices, Interrupts, and System Calls*).
3. **System calls**: the kernel boundary, with application code running unprivileged and requesting services through `svc` (same chapter).
4. **Memory protection**: static task partitions and the MPU, which stops a task from touching memory that is not its own (chapter *Memory Management*).
5. **Scheduling**: the periodic tick and the context switch let several tasks share the CPU (chapter *Scheduling*).

6. **Synchronization**: tasks that share data or devices coordinate their access, so that the result does not depend on where the tick falls (chapter *Synchronization*).